

Input & Controller System

Refactor Report — DUDE-GAME

Date: April 20, 2026 · Branch: main · Author: Claude Code

Overview

This report documents the input and controller system audit performed on the DUDE-GAME Unity project. Five C# scripts were reviewed and refactored. Issues fell into three categories: **performance** (unnecessary per-frame allocations and scene searches), **code quality** (duplication, dead code, debug leftovers), and **correctness** (array index mismatch that could cause runtime exceptions).

File	Changes	Category
PlayerInputHandler.cs	Extracted LinkComponents helper · removed duplicate GetComponent · removed debug prints	Code Quality
PlayerMovement.cs	Removed unused variable and unused import	Code Quality
PlayerCursor.cs	Cached PointerEventData, RaycastResult list, and PlayerCursor array · simplified CheckIfAllReady · added IsInitializedFor()	Performance
ControllerMapper.cs	Dirty-flag Update · InputSystem.onDeviceChange for disconnects · extracted helpers · fixed array length bugs · removed 4 unused imports	Performance + Bug Fix
GameController.cs	Removed per-frame string allocation from Update	Performance

1 · PlayerInputHandler.cs

Scripts/Inputs/PlayerInputHandler.cs

PROBLEM

- Awake() and reassignController() shared 8 identical lines of component-linking logic.
- reassignController() re-called GetComponent() even though playerInput was already cached in Awake.
- print() calls in OnInteract and OnReload left in production code.

FIX

- Extracted private LinkComponents(int targetIndex) helper. Both methods now call it — no duplication.
- Removed the redundant GetComponent call; playerInput is already available.
- Removed all print() debug calls from the two action handlers.

2 · PlayerMovement.cs

Scripts/Player/PlayerMovement.cs

PROBLEM

- Start() declared var input = GetComponent() and immediately discarded it — the variable was never read.
- using UnityEngine.InputSystem existed solely for that unused variable.

FIX

- Removed the unused variable assignment from Start().
- Removed the now-unnecessary using UnityEngine.InputSystem import.

3 · PlayerCursor.cs

Scripts/UI/PlayerCursor.cs

PROBLEM

- GetUIObjectUnderCursor() allocated new PointerEventData and new List on every call. Called every frame via HandleButtonHover — up to 240 heap allocations/second with 4 players.
- IsButtonHoveredByAnotherCursor(), CanStartGame(), and CheckIfAllReady() each called FindObjectsByType() independently with no caching.
- CheckIfAllReady() iterated the cursor list twice in two separate loops to compute the same result.

FIX

- PointerEventData created once in Awake and reused. List is a persistent field — cleared and reused each frame. Zero allocations in the hot path.
- PlayerCursor[] cached as a static field (_allCursors), populated in Awake, and reset on scene reload via the existing ResetStatics hook.
- CheckIfAllReady() reduced to a single pass counting both active and assigned in one loop.

Additionally, a public `IsInitializedFor(InputDevice)` method was added to support the `ControllerMapper` guard described below.

4 · ControllerMapper.cs

`Scripts/Managers/ControllerMapper.cs`

This was the highest-impact file. Four separate problems were addressed.

PROBLEM

- `Update()` called `initializeInputHandlers()` unconditionally every frame. That method runs `FindObjectsByType()` plus a LINQ `OrderBy` sort on every tick.
- The entire cursor assignment block (loop + `SetActive` calls) also ran every frame regardless of whether device state had changed.

FIX

- Added a bool `_dirty` flag. `Update` now calls `initializeInputHandlers()` and `UpdateCursorAssignments()` only when `_dirty` is true.
- Separated `DetectNewActivations()` — the only part that must poll every frame — from assignment logic that is now purely reactive.

PROBLEM

- Disconnected gamepads were detected via `RemoveAll` inside the `Update` loop, introducing one frame of lag and unnecessary polling.
- `PlayerCursor.Initialize()` was called every frame for already-initialized cursors, re-setting fields needlessly.

FIX

- Subscribed to `InputSystem.onDeviceChange` in `OnEnable/OnDisable`. Physical disconnections now set `_dirty = true` immediately via event.
- `UpdateCursorAssignments()` guards each `Initialize` call with `cursor.IsInitializedFor(device)` — skips the call entirely if the cursor is already bound to that device.

PROBLEM

- `EnableCursors()`, `DisableCursors()`, `EnablePlayerButtons()`, and `DisablePlayerButtons()` all looped over `playerInputHandlers.Length` while indexing into `playerCursors[]` or `playerButtons[]`. If array sizes differ, this throws `IndexOutOfRangeException`.
- Four unused imports: `System.Data`, `UnityEngine.Rendering`, `UnityEngine.SceneManagement`, `TMPPro`.

FIX

- All four methods now loop over their own array's `.Length` property.
- All four unused imports removed.

Two additional helpers were extracted for clarity: `HasSignificantInput (Gamepad)` isolates the `input-threshold` boolean, and `UpdateCursorAssignments()` replaces the mixed-responsibility `InitializeCursors()`. `InitializeControllerMapping()` was updated to call these directly.

5 · GameController.cs

Scripts/Managers/GameController.cs

PROBLEM • <code>Update()</code> called <code>print("aliveCount: " + aliveCount)</code> unconditionally, performing string concatenation and a heap allocation every single frame for the lifetime of every match.	FIX • Removed the call entirely. The value is observable in the Inspector via the serialized <code>aliveCount</code> field.
--	--

Remaining Architecture Debt

Two larger structural issues were identified during the audit. They were not auto-fixed because both require coordinated changes across most of the codebase and carry a higher risk of introducing regressions without a proper test pass.

Issue	Impact	Scope
Legacy Input API mixed with new Input System	Inconsistency between systems; actions cannot be rebound through the new Input System asset; harder to support new platforms.	<code>GameManager.Update (F11)</code> , <code>GameController.Update (L)</code> , <code>CardTransition (Space)</code>
<code>GameManager</code> uses flat per-player fields instead of arrays (<code>player1Score</code> , <code>player2Score...</code>)	Forces switch-case patterns in every file that reads player data. Adding or removing a player slot requires editing every switch block across the codebase.	<code>GameManager</code> , <code>PlayerCursor</code> , <code>PlayerInputHandler</code> , <code>GameController</code> — and any future system that reads player state.

Recommended approach: convert `GameManager` fields to arrays first (player data is the dependency), then migrate the legacy Input calls to Input Actions in a follow-up pass.